

1) What is this project?

Project Name

COPE SMS Admin System (Shaken Baby Alliance)

Problem we're solving

Caregivers text keywords like **HELP / CRY / SAFE / EMERGENCY** and the system replies with **guided, step-by-step support messages** (menus/options/remedies).

Admins need a **web dashboard** to create and manage these automated reply flows without coding.

What we are building

A **web-based Admin Console** + a **Flow Engine** that powers automated SMS replies.

- **Admin Console (UI):** create/edit/publish flows, templates, emergency rules, user directory, and view conversations.
- **Flow Engine (Backend logic):** matches keywords → sends step 1 → waits for reply → routes to next step → logs conversation.
- **SMS Integration (TBD):** receives incoming SMS webhook + sends outgoing SMS.

2) Core Features (Full list)

A) Admin Authentication + RBAC

- Login with **username/password**
- Roles:
 - **Viewer:** read-only
 - **Editor:** create/edit flows, templates
 - **Admin:** publish/disable + manage users
 - **Super Admin:** system settings
- No email auth
- No forgot password
- Admin creates users + assigns roles

B) Dashboard

- Stats cards: total flows, active flows, drafts, templates, emergency triggers, messages sent

- Recent activity table (audit)
- Quick buttons:
 - New Flow
 - New Template
 - Emergency Rules
 - Export

C) Flow Builder (Main Feature)

A “flow” is a guided script:

- Keyword triggers (HELP, CRY, etc.)
- Steps (Step 1, Step 2...)
- Step types:
 - Info
 - Menu options (reply 1/2/3)
 - Question
 - Emergency check
 - End
- Options map to next steps
- Fallback message for invalid replies
- Timeout support (optional)
- Publish / disable flows
- Test flow inside UI

D) Templates

Reusable messages:

- Greeting / fallback / emergency / disclaimer
- Used inside steps and emergency rules
- Activate/disable templates

E) Emergency Rules (No contact table)

- Keywords like: “911”, “bleeding”, “not breathing”
- On match: system sends **emergency template** containing contact details/instructions
- Logged as emergency event

F) Caregivers Directory (User table)

- Admin can add caregiver profile:
 - name, phone, email
 - baby info (name, DOB/age)
 - notes
- Unknown numbers auto-created as **unknown**
- Search by phone/email

G) Conversations & Messages

- Admin can view chat-style thread:
 - inbound user messages
 - outbound system replies
- Filter/search by phone/email
- See which flow + step was active when message happened

H) Audit Logs

- Track all admin actions:
 - flow created/edited/published
 - template changes
 - user added/disabled
- Export logs

I) Settings

- Session timeout

- Default fallback message
- Provider status (SMS service later)
- Export data (flows/templates/messages)

3) Tech Stack (What we will use)

Frontend (Admin UI)

You said: **Nuxt and Next.js**

✅ Best way: use **one** for admin UI to reduce complexity.

Option 1 (recommended): Nuxt for Admin UI

- Nuxt (UI)
- Nuxt server routes OR separate API service
- Nice for a single full-stack repo

Option 2: Next.js for Admin UI

- Next.js app router
- API routes for backend
- Also clean and popular

Pick one UI framework for the admin panel.

(You can still use Next.js later for a public website if needed.)

Backend

- API routes (REST endpoints)
- Webhook endpoint for incoming SMS (provider will call this)
- Flow engine service (route logic)

Database & ORM

- **Prisma ORM**
- **SQLite** (dev + small deploy)
- Later upgrade possible: Postgres

Containerization / Deployment

- **Docker**
- Docker Compose (optional)
- Runs:
 - web app
 - database (SQLite file volume)
 - optional worker service

Messaging provider (TBD)

You will decide later:

- Twilio (easy + common)
- AWS SNS (good if AWS)
- MessageBird / Vonage (optional)

4) What “Projects” means & How to do GitHub Projects (Step-by-step)

A) Create GitHub Project Board

Use **GitHub Projects (Table + Board)**

Columns/Views

- Backlog
- Sprint 1
- Sprint 2
- Sprint 3
- Sprint 4
- Testing/QA
- Done

Fields to add

- Sprint (Dropdown)
- Priority (High/Med/Low)
- Assignee

- Status
- Due date
- Component (UI / API / DB / SMS)

B) Create Epics (big tasks)

Create 6 epics as “Issues”:

- 1. Authentication + RBAC**
- 2. Admin UI Layout + Navbar**
- 3. Flow Builder + Flow Engine**
- 4. Templates + Emergency Rules**
- 5. Caregivers + Conversations**
- 6. Audit Logs + Settings + Export**

C) Create Sprint Tasks (small issues under epics)

Example tasks for Sprint 2 (UI + login)

- Build Login page UI
- Add role-based sidebar
- Create dashboard layout skeleton
- Set up Prisma schema + migrations
- Create admin_users CRUD endpoints

Each task becomes a GitHub Issue → added to project → assigned → sprint set.

5) Folder Structure (Clean for Nuxt/Next + Prisma)

If Nuxt full-stack (recommended)

```

/app
  /pages
  /components
  /layouts
  /middleware (auth guard)
/server
  /api
  /auth

```

/flows
/templates
/caregivers
/conversations
/prisma
 schema.prisma
/db
 sqlite.db
/docker
 Dockerfile
 docker-compose.yml

6) What will happen in real usage (End-to-end Flow)

1. Caregiver texts: HELP
2. SMS webhook hits backend /webhooks/sms
3. Backend:
 - checks emergency keywords
 - matches flow keyword HELP
 - sends Step 1 message
 - logs message
4. Caregiver replies 2
5. Backend routes to next step
6. Admin can view full conversation in dashboard

7) Deliverables (What you will submit)

- ✓ Admin UI (login + dashboard + flow builder + conversations)
- ✓ Database schema (Prisma + SQLite)
- ✓ Working flow engine logic
- ✓ Dockerized project
- ✓ Webhook ready for SMS provider (provider can be integrated later)

If you want, I can also send:

- ✓ **Project Charter (1–2 pages)**

- ☒ **Full sprint plan till April (8 sprints x 15 days)**
- ☒ **Final architecture diagram** for report (proper clean version)

Just tell me what you need next.

Awesome — you're using the **UTDallasEPICS Nuxt template (Better Auth + Prisma + SQLite)** as the base repo.

Below is the **FINAL folder structure** you should keep for this project **with all features mapped to exact folders/files** (so your team can implement cleanly).

☒ **Final Folder Structure (Nuxt + Prisma + SQLite) — COPE Admin System**

Goal: **Admin console + Flow Builder + Conversation Viewer + SMS Webhook + DB**

SMS provider is TBD, so we create a clean /webhooks/sms endpoint now.

1) Top-level structure (repo root)

shaken-baby-alliance-cope/

```
├─ app/           # UI (pages/components/layouts)
├─ server/        # Backend API (Nitro routes)
├─ prisma/        # Prisma schema + migrations + seed
├─ public/        # Static assets
├─ dockerfile     # Docker build file
├─ nuxt.config.ts
├─ package.json
├─ .env.example
├─ .gitignore
└─ README.md
```

☒ This matches your current repo and is correct.

2) app/ (Frontend UI) — final structure

app/

```
├─ layouts/
│   ├── default.vue    # Public layout (login)
│   └─ dashboard.vue   # Protected layout (sidebar + topbar)
├─ middleware/
│   └─ auth.global.ts  # redirects to /auth if not logged in
```

- └─ rbac.ts # helper to check role access in pages
- └─ pages/
 - └─ index.vue # Redirect: if logged in -> /dashboard else /auth
 - └─ auth/
 - └─ index.vue # Login page (username/password)
 - └─ dashboard/
 - └─ index.vue # Dashboard home (cards + activity)
 - └─ flows/
 - └─ index.vue # Flow list (search/filter/table)
 - └─ new.vue # Create flow form
 - └─ [id].vue # Flow Builder (edit flow + steps + options)
 - └─ templates/
 - └─ index.vue # Templates list
 - └─ new.vue # Create template
 - └─ [id].vue # Edit template
 - └─ emergency/
 - └─ index.vue # Emergency rules list + editor
 - └─ logs.vue # Emergency logs table
 - └─ caregivers/
 - └─ index.vue # Caregivers list (search phone/email)
 - └─ new.vue # Add caregiver manually
 - └─ [id].vue # Caregiver profile + linked conversations
 - └─ conversations/
 - └─ index.vue # Conversation list (search phone/email)
 - └─ [id].vue # Chat thread viewer (inbound/outbound)
 - └─ users/
 - └─ index.vue # Admin users management (Admin+)
 - └─ audit/
 - └─ index.vue # Audit logs (Admin+)
 - └─ settings/
 - └─ index.vue # System settings (Super Admin)
 - └─ help/

```

├── index.vue      # Help/Docs
├── components/
│   ├── nav/
│   │   ├── Sidebar.vue    # Sidebar links (role-based)
│   │   └── Topbar.vue     # Top bar (user, role, logout)
│   ├── dashboard/
│   │   ├── StatCard.vue   # Cards
│   │   └── ActivityTable.vue # Recent activity
│   ├── flows/
│   │   ├── FlowTable.vue
│   │   ├── FlowForm.vue
│   │   ├── StepList.vue
│   │   ├── StepEditor.vue
│   │   ├── OptionsEditor.vue
│   │   └── FlowPreview.vue # SMS preview + test panel
│   ├── conversations/
│   │   ├── ThreadView.vue # Chat bubbles
│   │   └── UserInfoCard.vue # caregiver details side panel
│   └── common/
│       ├── ConfirmModal.vue
│       ├── SearchBar.vue
│       └── DataTable.vue
├── composables/
│   ├── useAuth.ts      # currentUser, login/logout, role
│   ├── useRBAC.ts      # hasRole, canAccess helper
│   └── useApi.ts       # typed API wrapper
├── assets/
└── styles.css

```

3) server/ (Backend API routes) — final structure

All backend endpoints live here.

Naming format: route.get.ts, route.post.ts, etc.

```

server/
├── api/

```

```

├─ auth/
│   ├── login.post.ts    # username/password login
│   ├── logout.post.ts   # destroy session
│   └─ me.get.ts        # current admin user
├─ admin-users/
│   ├── index.get.ts     # list admin users (Admin+)
│   ├── index.post.ts    # create admin user (Admin+)
│   ├── [id].put.ts      # update role/status (Admin+)
│   └─ [id].delete.ts    # disable user (Admin+)
├─ flows/
│   ├── index.get.ts     # list flows (filters)
│   ├── index.post.ts    # create flow
│   ├── [id].get.ts      # flow details
│   ├── [id].put.ts      # update flow
│   ├── [id].delete.ts   # disable flow (Admin+)
│   └─ [id]/publish.post.ts # publish flow (Admin+)
├─ flow-keywords/
│   ├── index.post.ts    # add keyword to flow
│   └─ [id].delete.ts    # remove keyword
├─ steps/
│   ├── index.post.ts    # create step
│   ├── [id].put.ts      # update step
│   └─ [id].delete.ts    # delete step
├─ step-options/
│   ├── index.post.ts    # add option to step
│   ├── [id].put.ts      # update option
│   └─ [id].delete.ts    # delete option
├─ templates/
│   ├── index.get.ts     # list templates
│   ├── index.post.ts    # create template
│   ├── [id].get.ts
│   ├── [id].put.ts
│   └─ [id].delete.ts
├─ emergency/
│   ├── rules.get.ts     # list emergency rules
│   ├── rules.post.ts    # create rule
│   └─ rules/[id].put.ts # update rule

```

```

├─ rules/[id].delete.ts # delete rule
├─ logs.get.ts          # emergency logs
├─ caregivers/
│   ├─ index.get.ts     # list caregivers (search phone/email)
│   ├─ index.post.ts    # add caregiver
│   ├─ [id].get.ts      # caregiver profile
│   ├─ [id].put.ts      # update caregiver
│   └─ [id].delete.ts   # block/disable caregiver
├─ conversations/
│   ├─ index.get.ts     # list conversations
│   └─ [id].get.ts      # conversation thread + messages
├─ messages/
│   └─ [conversationId].get.ts # messages for conversation (pagination)
├─ dashboard/
│   └─ stats.get.ts     # cards + recent activity
├─ audit/
│   └─ index.get.ts     # audit logs list
├─ webhooks/
│   └─ sms.post.ts      # provider webhook (TBD)
├─ utils/
│   ├─ db.ts           # Prisma client singleton
│   ├─ auth.ts         # auth helpers (requireUser, role check)
│   ├─ rbac.ts         # permission rules
│   ├─ flow-engine.ts   # main flow routing logic
│   ├─ sms-provider.ts  # abstraction (twilio/sns later)
│   └─ validators.ts    # zod schemas / input validation
├─ plugins/
│   └─ prisma.ts        # optional prisma injection

```

✅ Key idea: keep **flow-engine** logic separate so webhook + testing both reuse it.

4) prisma/ (Database) — final structure

```

prisma/
├─ schema.prisma      # your final DB schema

```

└─ migrations/ # generated by Prisma
└─ seed.ts # seed default admin + sample flows

✅ Seed should create:

- super_admin user
- 1 sample flow (HELP)
- 1 emergency template
- 1 emergency rule

5) Features → File Mapping (So you don't miss anything)

Authentication + RBAC

- UI: app/pages/auth/index.vue
- Middleware: app/middleware/auth.global.ts, app/middleware/rbac.ts
- API: server/api/auth/*, server/utils/auth.ts, server/utils/rbac.ts

Dashboard

- UI: app/pages/dashboard/index.vue
- API: server/api/dashboard/stats.get.ts

Flow Builder

- UI: app/pages/flows/*, app/components/flows/*
- API: server/api/flows/*, server/api/steps/*, server/api/step-options/*
- Engine: server/utils/flow-engine.ts

Templates

- UI: app/pages/templates/*
- API: server/api/templates/*

Emergency rules + logs

- UI: app/pages/emergency/*
- API: server/api/emergency/*

Caregivers Directory

- UI: app/pages/caregivers/*
- API: server/api/caregivers/*

Conversations + messages

- UI: app/pages/conversations/*
- API: server/api/conversations/*, server/api/messages/*

Audit Logs

- UI: app/pages/audit/index.vue
- API: server/api/audit/index.get.ts

SMS Webhook (Provider TBD)

- API: server/api/webhooks/sms.post.ts
- Provider layer: server/utls/sms-provider.ts

6) What you should change in your current template README

Because your template mentions OTP/email, but you don't want it.

Update README to say:

- Auth = **username/password** (no email OTP)
- SMS provider = TBD
- Admin creates users manually

7) “Final Answer” summary (what should be in the repo)

- ✓ Nuxt UI pages for all modules
- ✓ server APIs for all modules
- ✓ Prisma schema + migrations + seed
- ✓ Dockerfile for deployment
- ✓ Webhook endpoint ready for provider integration

1) Admin UI Screens You Need

- ✓ A) Keywords / Triggers

Admins define what incoming text triggers which Flow.

- Keyword: HELP
- Also aliases: HELP, Help, help me, assist, support
- Routes to: HELP_FLOW_EN or HELP_FLOW_ES

✔ B) Flow Editor

Admins create the multi-step chain (steps + options + links).

✔ C) Templates (optional)

If you store message text directly inside steps, you can skip templates initially.

✔ D) Test Simulator (very useful)

Admin can “simulate chat” and see if the chain works before publishing.

2) Proper Flow Editor UI (Wireframe Diagram)

This design is **simple + scalable**.

```
=====
FLOW EDITOR: HELP_FLOW_EN                Status: Draft (●)
-----
[Back] Flow Name: [ HELP Flow ] Language: [EN ▼] Trigger: [HELP ▼]
      Start Step: [Step 1 ▼] Active: [ ON ] Version: v3
-----
LEFT PANEL (Steps)          RIGHT PANEL (Selected Step Editor)
-----
Steps          Step Details
|-----|-----|
| Step 1 (Start) | | Step Name: [Step 1 - Welcome] |
| Step 2 (Menu)  | | Step Type: [ MENU ▼ ] |
| Step 3 (Crying - Basics) | | Message Text (SMS): |
| Step 4 (Crying - Remedies) | | |-----| |
| Step 5 (Emergency) | | |"I'm here with you. Choose:" |
| ... | | 1 Baby crying 2 Remedies |
| + Add Step | | 3 Talk to someone 4 Emergency |
|-----|-----|
| |
| Options (User Replies → Next) |
| |-----| | |
| |Input| Action | Next |
| |1 | Go to Step | Step 3 |
| |2 | Go to Step | Step 4 |
| |3 | SwitchFlow | HUMAN_FLOW |
```

```

|| 4 | Go to Step | Step 5 ||
|| MENU | Go to Step | Step 2 ||
| _____ |
| [ + Add Option ] |
| _____ |
| [ Save Step ] [ Delete Step ] |
|_____

```

Bottom Bar: [Run Test Simulator] [Save Draft] [Publish] [View Diagram]
=====

Why this works for 10+ steps

- Steps list stays organized
- Each step has its own message + options
- You can add unlimited steps and link them

3) How Admin Adds: Keyword = HELP → Multi-step Chain

A) In “Keywords / Triggers” screen

Create Trigger

- Trigger Name: HELP Keyword
- Match Type: Exact + Contains (MVP can be “contains”)
- Keywords/Aliases: help, help me, support, assist
- Route To Flow: HELP_FLOW_EN (or language-based routing)

✅ Now any user text containing “help” can start that flow.

B) In “Flow Editor”: Build the chain

✅ **Step 1 (Start): Welcome + language-safe default**

Message Text

“Hi, you’ve reached COPE Text Support. I can help.

Reply **1** for options, or text **MENU** anytime.”

Options

- Input: 1 → Go to Step → Step 2
- Input: MENU → Go to Step → Step 2

✅ Step 2: HELP Menu (the hub)

Message Text

“How can I help right now?”

Reply:

1 Baby crying intensely

2 Remedies / calming steps

3 Talk to a support person

4 Emergency / I feel unsafe”

Options

- 1 → Step 3 (Crying - Basics)
- 2 → Step 4 (Remedies)
- 3 → Switch Flow → HUMAN_SUPPORT_FLOW
- 4 → Step 5 (Emergency)

✅ This is where the “4 options under HELP” lives.

✅ Step 3: Baby Crying Intensely (Basics)

Message Text

“Crying can be extremely stressful. Start here:

1. Check diaper/feeding/temperature
2. Reduce noise/lights
3. Gentle rocking + steady breathing

Reply **1** for more calming, **2** to go back to menu, **4** emergency.”

Options

- 1 → Step 4 (Remedies)
- 2 → Step 2 (Menu)
- 4 → Step 5 (Emergency)

✅ Step 4: Remedies / Calming Steps (more detailed)

Message Text

“Try one of these for 2 minutes:

A) Safe place (crib) + 10 slow breaths

B) Walk to another room, drink water

C) White noise / gentle sway

Reply **A, B, C,** or **MENU.**”

Options

- A → Step 6 (Check-in after A)
- B → Step 7 (Check-in after B)
- C → Step 8 (Check-in after C)
- MENU → Step 2

✅ Now you see how it naturally becomes 10+ steps.

✅ Step 5: Emergency (global safety step)

Message Text

“Please place the baby in a safe place (crib) now and step away.

If anyone is in immediate danger, call **911**.

Reply **SAFE** when the baby is safe.”

Options

- SAFE → Step 9 (Post-emergency check-in)
- MENU → Step 2

✅ Backend action on this step: create escalation record.

4) How Admin Handles “Some keywords have 3 steps, some have 10+”

✅ Each keyword routes to a Flow

- HELP → HELP_FLOW
- STRESS → STRESS_FLOW
- CRYING → CRYING_FLOW
- RESOURCES → RESOURCES_FLOW

✓ **Each flow can have any number of steps**

- 3 steps = just create 3
- 10+ steps = keep adding steps and linking options

✓ **Keep it clean with “Hub Menu” pattern**

For large flows, use:

- Step 2 = “Hub Menu”
- everything returns to Step 2 via MENU

That prevents getting lost.

5) Editor Features You MUST include (so admins don’t break flows)

Validation rules

- Every flow must have **Start Step**
- Every option must have a valid destination (step/flow)
- Prevent duplicate option inputs in the same step (1 twice)
- Show warning if a step has **no way out** (dead end)
- “Emergency option” should always exist or be keyword override

Test Simulator (strongly recommended)

Admin types:

- “help”
- then “1”
- then “A”
and UI shows the bot replies instantly (no Twilio needed).

6) What to build FIRST (MVP editor)

If you want the easiest working editor:

- ✓ Flow list (create/edit)
- ✓ Step list + Step editor form
- ✓ Options table + Add option
- ✓ Set start step

✓ Publish toggle

✓ Test simulator (basic)

That's enough to support 10+ step chains.

If you want, I can also send a **“HELP Flow as a diagram”** like this (boxes + arrows) so you can paste it into docs:

HELP → Step 2 Menu

1 → Step 3 Crying → 1 → Step 4 Remedies → A/B/C → Step 6/7/8

2 → Step 4 Remedies ...

3 → Human Flow

4 → Step 5 Emergency → SAFE → Step 9

Admin Authentication & Access Control Plan

1 Overview

This project includes a secure **Admin Dashboard** used only by authorized client personnel.

Caregivers do **not** log in to the system. They interact only via SMS.

Access to the dashboard is restricted through **admin-managed authentication and role-based access control (RBAC)**.

2 Initial System Setup (Super Admin)

At system deployment, a **Super Admin account** will be created manually (via seed script or direct database entry).

Super Admin Capabilities

The Super Admin has full system access, including:

- Creating new admin users
- Assigning roles (Viewer / Editor / Admin)
- Modifying user access
- Deactivating users
- Managing flows, steps, and triggers
- Publishing conversation flows
- Viewing and managing escalations

- Accessing all system settings

Super Admin Creation Method

- Username and password are configured during initial setup.
- Password is securely hashed (bcrypt/argon2).
- No public signup is available.

Admin User Management

Admin accounts are **not self-registered**.

All additional admin users are created by an existing Admin (or Super Admin) from the Admin Dashboard.

Admin User Creation Flow

1. Super Admin logs in.
2. Navigates to **Admin Users** section.
3. Clicks **Add User**.
4. Enters:
 - Username
 - Password
 - Role (Viewer / Editor / Admin)
 - Active status
5. Saves user.
6. Credentials are shared securely with the intended user.

Role-Based Access Control (RBAC)

The system supports three role levels:

Viewer

- Can view Inbox and Conversations
- Can view Flows and Steps
- Can view Escalations
- Cannot edit or modify content

- Cannot create users

🔧 Editor

- Can create and edit Flows
- Can create and modify Steps and Options
- Can manage Keyword Triggers
- Can publish flows
- Cannot manage admin users
- Cannot modify system-level settings

🛡️ Admin

- Full system access
- Can manage all flows and triggers
- Can create, edit, deactivate admin users
- Can manage escalations
- Can configure system settings

5 Authentication Model

The system uses **simple username + password authentication** for admin access.

There is:

- ✅ Login page
- ❌ No public registration
- ❌ No email verification
- ❌ No forgot password option (MVP phase)

Passwords are securely hashed in the database.

6 No “Forgot Password” Feature (MVP Decision)

For this project phase:

- The login page will NOT include a "Forgot Password" option.
- If a password needs to be changed:
 - An Admin can manually reset it in the Admin Dashboard.
 - The new password will be securely hashed and updated.

This keeps the system simple and controlled since:

- Only internal client users access the dashboard.
- There is no public-facing login.

7 Security Controls

The following controls will be implemented:

- Password hashing (bcrypt/argon2)
- Role-based API protection (middleware)
- Route-level permission enforcement
- Account activation/deactivation control
- Prevention of unauthorized dashboard access
- Session/token validation for all admin routes

8 API Protection Strategy

Each API endpoint will define minimum required role level:

API	Minimum Role
View Inbox	Viewer
Edit Flow	Editor
Publish Flow	Editor
Create Admin User	Admin
Delete Admin User	Admin
System Settings	Admin

Backend middleware enforces these rules.

🔗 Summary of Admin Architecture

The Admin Access System consists of:

- One initial Super Admin account
- Admin-managed user provisioning
- Three role levels (Viewer, Editor, Admin)
- No public signup
- No forgot password functionality (MVP)
- Secure password hashing
- Backend role-based access enforcement

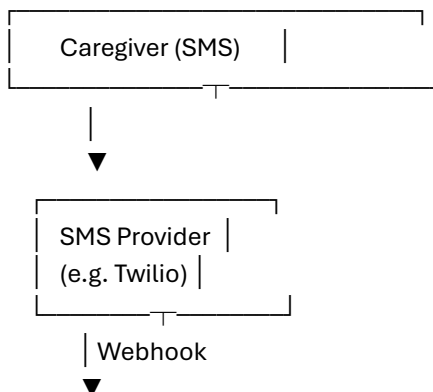
This ensures:

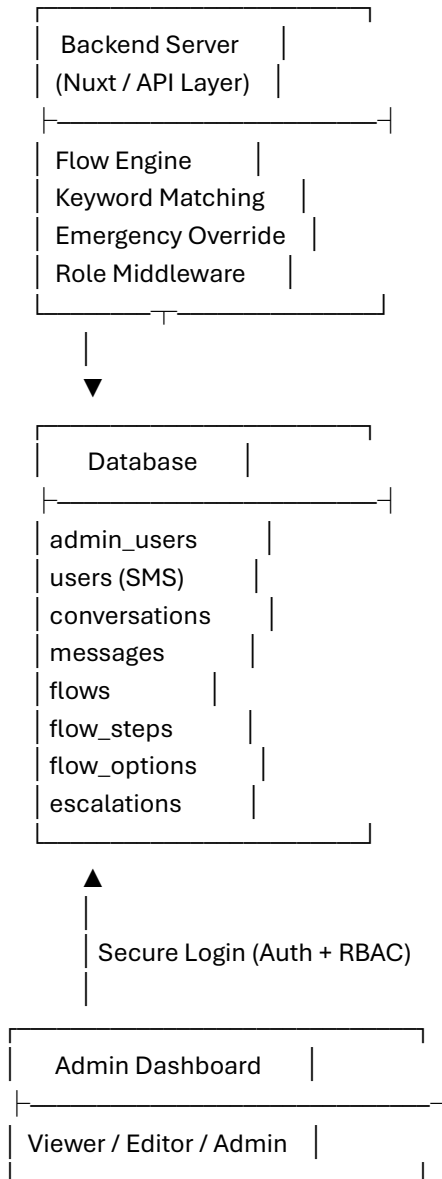
- Controlled access
- Simple administration
- Secure role separation
- Clear operational boundaries

Perfect — below is a **visual architecture diagram section** you can directly include in your project plan document.

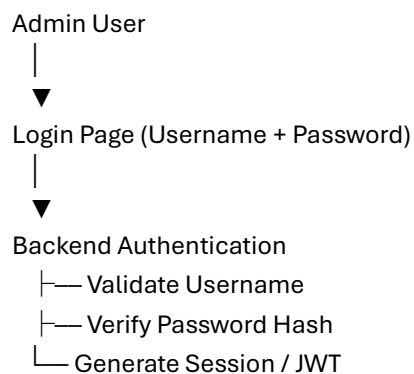
🔗 Admin Authentication & Access Architecture Diagram

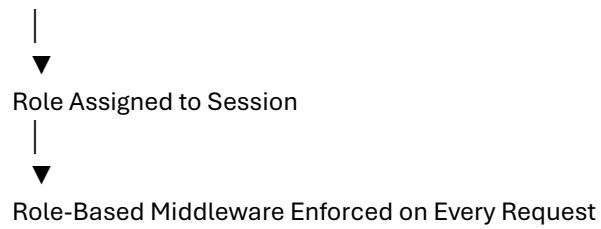
🔗 1 High-Level System Architecture



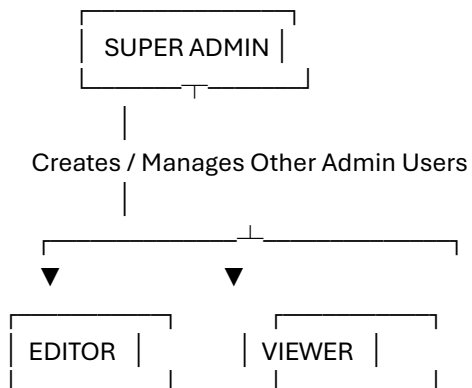


2. Admin Authentication Flow





🛡️ 3️⃣ Role-Based Access Control (RBAC) Model



Role Hierarchy (Numeric Model)

Viewer = 1

Editor = 2

Admin = 3

Permission rule:

If user.role_level >= required_level

→ Allow

Else

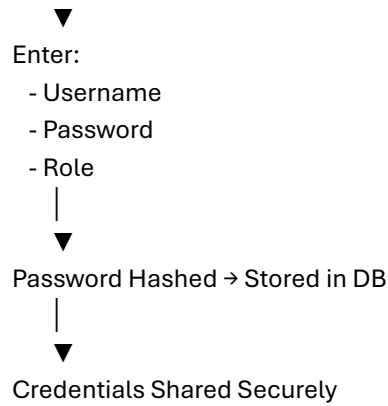
→ 403 Forbidden

🧩 4️⃣ Admin User Management Flow

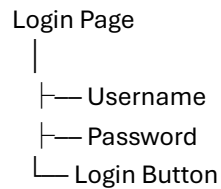
Super Admin Login

Admin Dashboard → Admin Users Page

+ Create User



🚫 5️⃣ No Forgot Password Flow (MVP Decision)



(No Forgot Password Link)

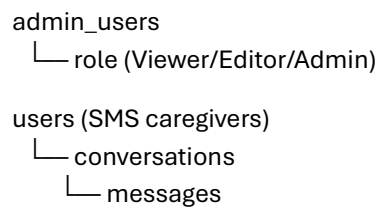
If password reset needed:

Admin → Edit User → Set New Password → Save

🔒 6️⃣ Security Enforcement Layers

- Layer 1: Login Authentication
- Layer 2: JWT / Session Validation
- Layer 3: Role-Based Middleware
- Layer 4: API Route Protection
- Layer 5: UI Button Restrictions (Frontend)
- Layer 6: Optional Audit Logging

📦 7️⃣ Database Relationship Overview (ER Style)



```
flows
├── flow_steps
└── flow_options
```

```
conversations
├── current_flow_id
└── current_step_id
```

```
conversations
└── escalations
```

Final Summary Diagram (System Control Flow)

```
graph TD
    A[Caregiver SMS] --> B[SMS Webhook]
    B --> C[Flow Engine]
    C --> D["Database (Flows + Steps + Options)"]
    D --> E[Reply Sent]
    F[Admin Login] --> G[Role Check]
    G --> H[Dashboard Access]
    H --> I[Flow Management / User Management]
```

This completes the **visual admin authentication and access architecture section** for your project documentation.

If you want, I can now generate a **combined master system architecture diagram** that includes:

- SMS Flow Engine
- Admin Access
- Database Structure
- Role Model
- Emergency Escalation Flow

All in one clean diagram.

Got it — you wan  **A) APP NAVIGATION TREE (Every page under sidebar)**

LOGIN

- └─ Dashboard
 - └─ Flows
 - └─ Flow List
 - └─ Create Flow
 - └─ Flow Builder (Edit Flow)
 - └─ Flow Versions (optional but recommended)
 - └─ Templates
 - └─ Template List
 - └─ Create/Edit Template
 - └─ Emergency
 - └─ Emergency Rules
 - └─ Emergency Templates (optional link)
 - └─ Emergency Logs
 - └─ Contacts
 - └─ Groups
 - └─ Contacts List
 - └─ Imports (CSV)
 - └─ Users & Roles (Admin+)
 - └─ Users
 - └─ Roles (fixed)
 - └─ Permissions matrix (view)
 - └─ Audit Logs (Admin+)
 - └─ Settings (Super Admin)
 - └─ System
 - └─ SMS Provider
 - └─ Security
 - └─ Data Export
 - └─ Maintenance
 - └─ Help / Docs

B) GLOBAL LAYOUT (Shared on every page after login)

COPE Admin

[Role Badge] [Logout]

Sidebar	Page Header: Title + Breadcrumb + Actions (top-right)
- Dashboard	
- Flows	Page Content
- Templates	
- Emergency	
- Contacts	
- Users&Roles	
- Audit Logs	
- Settings	
- Help	

Global Header Buttons

- Profile menu: My Profile (optional), Logout
- Role badge display
- Optional: “Environment” badge (Dev/Prod)

✅ C) DASHBOARD — FULL DETAILS (Sections + Buttons + Tables)

Dashboard Wireframe (Full)

Dashboard	[Refresh]
QUICK STATS (Cards)	
Total Flows	Active Flows
24	18
Draft Flows	Disabled Flows
4	2
Templates	Emergencies
12	3 (7d)
Messages Sent	Active Users
120 (7d)	6
QUICK ACTIONS	
[+ New Flow]	[+ New Template]
[Emergency Rules]	[Export Data]
SYSTEM STATUS	
SMS Provider: [Connected ✅]	DB: [Healthy ✅]
Last Backup: [Yesterday]	
RECENT ACTIVITY (Table)	

Time	User	Action	Target	Result		

10:10AM	Admin	Published Flow	HELP Flow	Success		
09:55AM	Editor	Edited Step	CRY Flow	Saved Draft		

ALERTS (if any)	
-----------------	--

⚠ Draft flows pending publish: 2	[View Drafts]		
⚠ Emergency triggers today: 1	[View Logs]		

Dashboard Buttons (Exactly)

- Refresh
- + New Flow
- + New Template
- Emergency Rules
- Export Data
- View Drafts
- View Logs

✅ D) FLOWS — FULL UI (List + Create + Builder + Versions)

D1) Flows List Page (Everything)

Flows						[+ New Flow]	[Export]	
Search: [] Status: [All ▼] Category:[All ▼] Sort:[▼]								
Name	Keywords	Category	Status	Updated	Actions			

HELP Flow	HELP,HELPP	Support	Active	Today	View Edit			
CRY Flow	CRY	Crying	Draft	2 days	View Edit			
EMERG Flow	911,BLEED	Emergency	Active	Today	View			

Bulk Actions: [Select All] [Disable] [Duplicate] (Admin/Editor)

Buttons / Actions

- Top right: + New Flow, Export
- Per row:
 - View
 - Edit (Editor+)
 - Duplicate (Editor+)
 - Disable (Admin+)
- Filters: Status / Category / Sort
- Bulk: Disable / Duplicate

D2) Create Flow Page (Minimal but complete)

Fields:

- Flow Name
- Category
- Trigger Keywords (comma separated)
- Default Language (optional)
- Start Step (auto Step 1)

Buttons:

- Create Draft
- Cancel

D3) Flow Builder (FULL detail, all buttons)

Flow Builder: HELP Flow		Status: [Draft ▼] [Save] [Publish] [X]	
LEFT: Outline		CENTER: Step Editor	RIGHT: Preview/Test
Flow Details	Step: [Step 1 ▼] (Order: 1)		SMS Preview
- Name	Title: [Baby crying intensely]		Baby crying?
- Category	Type: [Menu Options ▼]		

- Keywords	Message Text:	Reply:	
		1 Swaddle	
Steps List	"Try these steps..."	2 Feed	
[Step 1]		3 Call help	
[Step 2]	Char Count: 124/160		
[Step 3]			
[+ Add Step]	Options (Max 6):	Validation	
[Reorder]	1) Label:[Swaddle] Next:[Step 2 ▼]	☒ No broken links	
	2) Label:[Feed] Next:[Step 3 ▼]	☒ Options valid	
	3) Label:[Emergency] Next:[END ▼]	☐ Long SMS (if any)	
Step Rules	Invalid Reply Msg:	Test Simulator	
- Fallback	[Please reply 1-3]	User Reply: [2]	
- Timeout	Timeout (mins): [5]	[Send] [Reset]	
	Timeout Msg: [Are you there?]		
	Versioning		
	Step Actions:	[View Versions]	
	[Duplicate Step] [Delete Step]		

Flow Builder Buttons (Complete list)

- Top: Save, Publish (Admin), Close
- Left panel: + Add Step, Reorder
- Step actions: Duplicate Step, Delete Step
- Right panel: Test Send, Reset
- Versions: View Versions
- Status dropdown: Draft / Active / Disabled

D4) Flow Versions (Recommended)

Purpose: track changes + rollback.

Table columns:

- Version #
- Created by
- Created at
- Notes
- Buttons: View, Restore (Admin+)

✓ E) TEMPLATES — FULL UI

Template List

Buttons: + New Template, Export

Columns:

- Name
- Type (Greeting, Fallback, Emergency, Disclaimer)
- Status
- Updated
- Actions: View/Edit/Duplicate/Disable

Template Editor

Fields:

- Name
 - Type dropdown
 - Body text
 - Variables insert (optional)
- Buttons:

- Save
- Activate/Disable (Admin)
- Delete (Admin)

✓ F) EMERGENCY — FULL UI

F1) Emergency Rules

Keyword | Match Type (exact/contains) | Action (Template/Escalate/Both) | Active | Edit

Buttons:

- + Add Rule
- Save
- Test Keyword

F2) Emergency Logs

Filters:

- Date range
- Trigger keyword
- Action type

Table:

- Time
- Trigger
- Flow context
- Action taken
- Notified group
- Status

G) CONTACTS — FULL UI

Tabs:

- Groups
- Contacts
- Import

Groups

Fields:

- Group name
 - Description
- Buttons:

- Add
- Edit
- Delete

Contacts

Fields:

- Name
- Phone (or email)

- Group
 - Status
- Buttons:
- Add contact
 - Disable contact
 - Bulk import

Import

- Upload CSV
- Mapping UI (column mapping)
- Validate
- Import

✓ H) USERS & ROLES — FULL UI (Admin+)

Users List

Buttons:

- + Create User
- Export Users

Columns:

- Username
- Role
- Status
- Last login
- Actions: Reset Password, Disable, Change Role

Create User Modal

- Username
 - Temp password
 - Role dropdown
 - Force change password on first login (optional)
- Buttons: Create / Cancel

✓ I) AUDIT LOGS — FULL UI (Admin+)

Filters:

- User
- Action type
- Date range

Table:

- Timestamp
- User
- Action
- Object
- Before/After (view details)

Button:

- Export logs

✓ J) SETTINGS — FULL UI (Super Admin)

Tabs:

1. System
2. SMS Provider
3. Security
4. Data Export
5. Maintenance

System

- App name
 - Default language
 - Default fallback template
- Buttons: Save

SMS Provider

- Provider: AWS SNS / Twilio

- Provider status (connected/not)
- Sender ID / Phone
Buttons: Test SMS / Save

Security

- Session timeout minutes
- Rate limiting (on/off)
- Password policy (min length)
Buttons: Save

Data Export

- Export flows JSON
- Export templates
- Export logs
Buttons: Export

Maintenance

- Maintenance mode on/off
- Backup now
Buttons: Enable / Backup

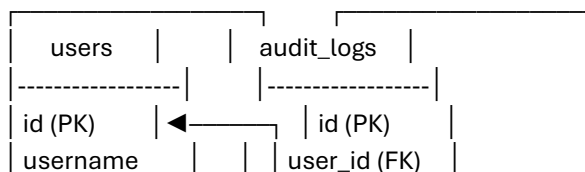
✓ K) HELP / DOCS

Sections:

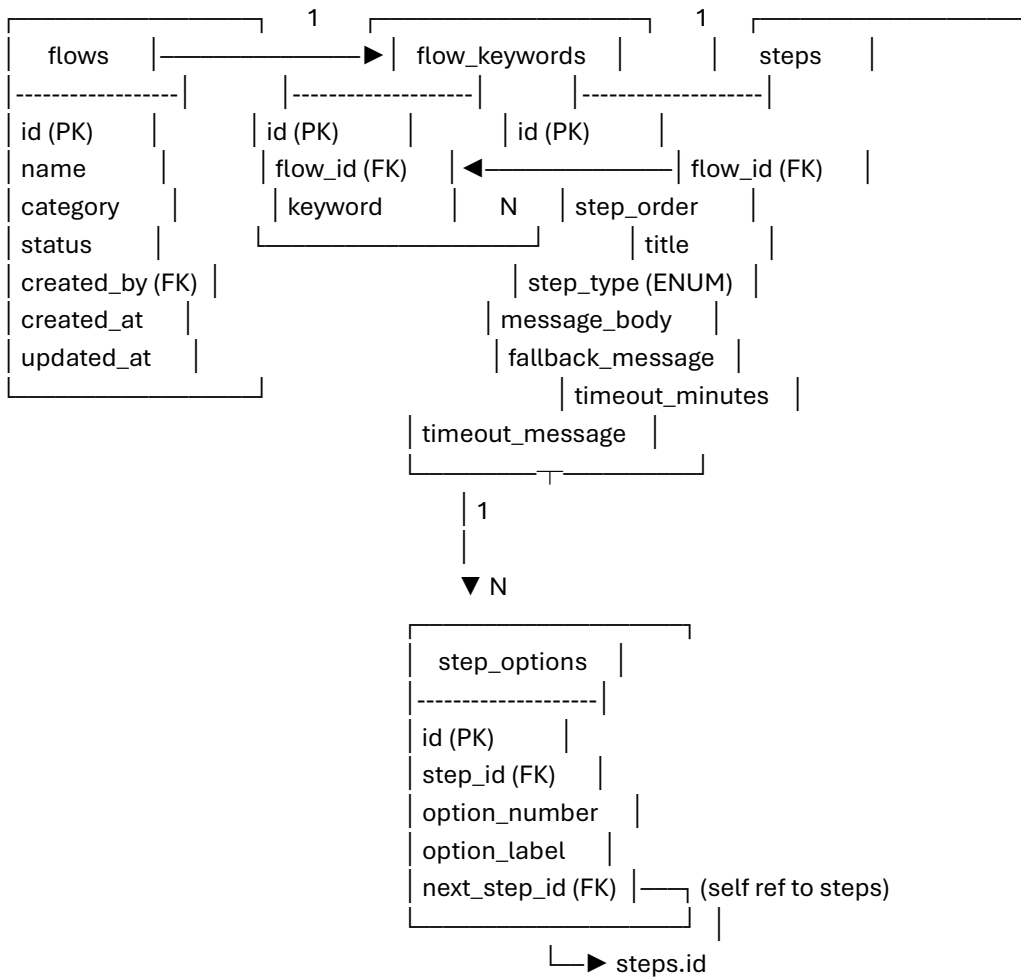
- How keywords work
- How flows work (steps/options)
- Best practices for SMS length
- Emergency guidelines

✓ L) FULL ER DIAGRAM (Visual Layout – Detailed)

Here is the **full ER diagram wireframe** (like your screenshot style):



password_hash	└─▶	action
role (ENUM)		object_type
status (ENUM)		object_id
last_login		before_json
created_at		after_json
		created_at



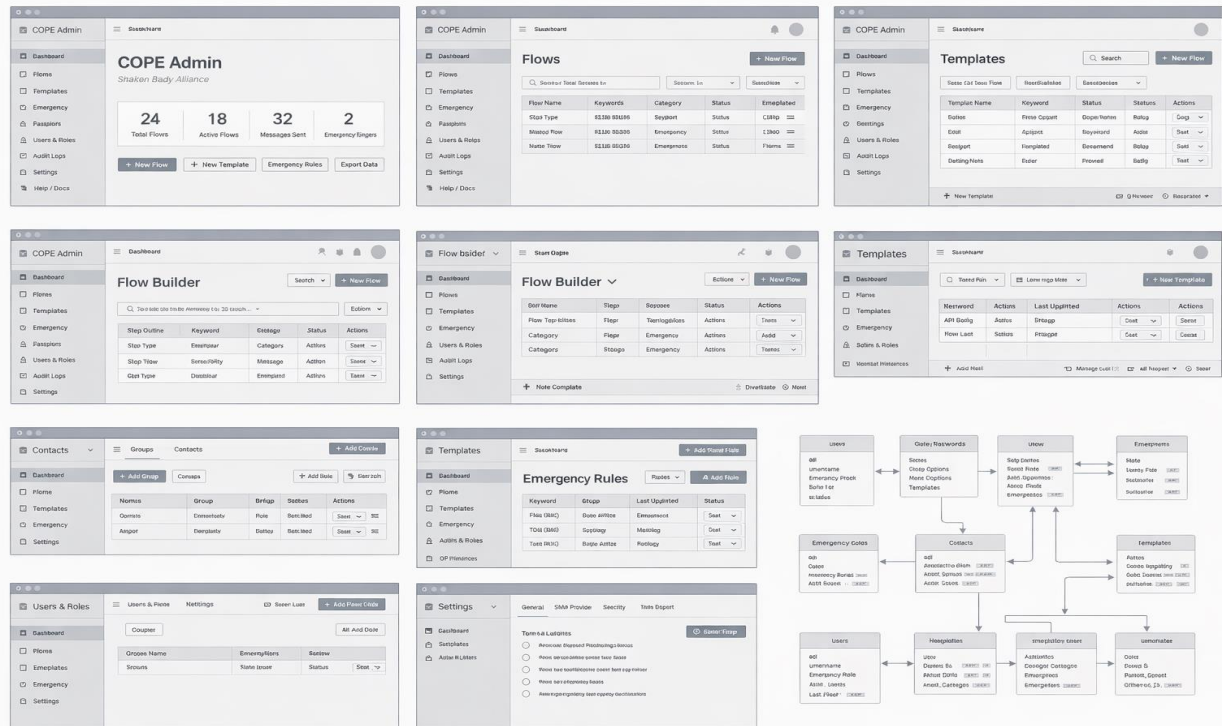
templates

id (PK)
name
type (ENUM)
body
status (ENUM)
created_by (FK)
created_at

updated_at	
------------	--

emergency_rules	emergency_logs
-----	-----
id (PK)	id (PK)
keyword	triggered_keyword
match_type	matched_rule_id(FK)
action_type	flow_id (FK, null)
template_id (FK)	action_taken
contact_group_idFK	created_at
is_active	

contact_groups	1	contacts
-----	→	-----
id (PK)		id (PK)
group_name		name
description		phone
is_active		group_id (FK)
		status (ENUM)



COPE SMS Admin System — Final Database Architecture (No Contacts Tables)

0) Relationship Map (Final)

Admin / Content

- admin_users → creates/updates → flows, templates, emergency_rules, caregivers
- flows → has many → flow_keywords
- flows → has many → flow_steps
- flow_steps → has many → step_options
- step_options → routes to → flow_steps (self FK next_step_id)
- templates used by → step_options, emergency_rules

Caregiver / Conversation History

- caregivers → has many → conversations
- conversations → has many → messages

- messages optionally references → flows, flow_steps (context)

Auditing

- audit_logs references → admin_users (+ objects)

✓ Removed tables

- contact_groups
- contacts

✓ Emergency contact details will be stored in:

- templates.body (emergency template includes phone numbers)
- and/or emergency_rules.emergency_contact_text

1) Admin Authentication Tables

1.1 admin_users

Joins / Keys

- admin_users.id → joins with:
 - flows.created_by_admin_id, flows.updated_by_admin_id
 - templates.created_by_admin_id, templates.updated_by_admin_id
 - audit_logs.admin_user_id
 - caregivers.created_by_admin_id (optional)

Purpose

Admin login accounts + RBAC.

Column	Type	Key	Why
id	UUID	PK	Unique admin id
username	VARCHAR	UNIQUE	Login id
password_hash	VARCHAR		Secure password storage
role	ENUM		viewer/editor/admin/super_admin
status	ENUM		active/disabled

last_login_at	TIMESTAMP NULL	Security tracking
created_at	TIMESTAMP	Tracking
updated_at	TIMESTAMP	Tracking

Indexes

- UNIQUE(username)

1.2 admin_sessions (Optional)

Joins / Keys

- admin_sessions.user_id → FK to admin_users.id

Column	Type	Key	Why
id	UUID	PK	Session id
user_id	UUID	FK	Which admin
session_token_hash	VARCHAR		Secure cookie session
expires_at	TIMESTAMP		Auto expiry
revoked_at	TIMESTAMP NULL		Force logout
created_at	TIMESTAMP		Tracking

2) Flow Builder Tables (Core)

2.1 flows

Joins / Keys

- flows.id → joins with:
 - flow_keywords.flow_id
 - flow_steps.flow_id
 - conversations.current_flow_id
 - messages.flow_id
- created_by_admin_id, updated_by_admin_id → FK to admin_users.id

Column	Type	Key	Why
id	UUID	PK	Unique flow
name	VARCHAR		UI display name
category	VARCHAR		Filter/group
status	ENUM		draft/active/disabled
description	TEXT NULL		Notes
start_step_id	UUID NULL	FK → flow_steps.id	Entry step
created_by_admin_id	UUID	FK	Owner
updated_by_admin_id	UUID	FK	Last editor
published_at	TIMESTAMP NULL		When activated
created_at	TIMESTAMP		Tracking
updated_at	TIMESTAMP		Tracking

Indexes

- index(status), index(category)

2.2 flow_keywords

Joins / Keys

- flow_keywords.flow_id → FK to flows.id

Column	Type	Key	Why
id	UUID	PK	Keyword row
flow_id	UUID	FK	Parent flow
keyword	VARCHAR		Trigger word
is_active	BOOLEAN		Disable keyword
created_at	TIMESTAMP		Tracking

Constraints

- UNIQUE(flow_id, keyword)

Indexes

- index(keyword), index(flow_id)

2.3 flow_steps

Joins / Keys

- flow_steps.flow_id → FK to flows.id
- step_options.step_id → FK to flow_steps.id
- step_options.next_step_id → FK to flow_steps.id (self ref)
- conversations.current_step_id → FK
- messages.step_id → FK (context)

Column	Type	Key	Why
id	UUID	PK	Step id
flow_id	UUID	FK	Parent flow
step_order	INT		Step 1/2/3
title	VARCHAR		Admin label
step_type	ENUM		info/menu/question/emergency_check/end
message_body	TEXT		Message to send
fallback_message	TEXT NULL		Invalid reply
timeout_minutes	INT NULL		Timeout feature
timeout_message	TEXT NULL		Timeout message
created_at	TIMESTAMP		Tracking
updated_at	TIMESTAMP		Tracking

Constraints

- UNIQUE(flow_id, step_order)

2.4 step_options

Joins / Keys

- step_options.step_id → FK to flow_steps.id
- step_options.next_step_id → FK to flow_steps.id (nullable)
- step_options.template_id → FK to templates.id (nullable)

Column	Type	Key	Why
id	UUID	PK	Option id
step_id	UUID	FK	Parent step
option_number	INT		Reply number (1..6)
option_label	VARCHAR		Shown to user
action_type	ENUM		route_to_step/send_template/end/emergency
next_step_id	UUID NULL	FK	Routing
template_id	UUID NULL	FK	If sending template
created_at	TIMESTAMP		Tracking

Constraints

- UNIQUE(step_id, option_number)

3) Templates Table

3.1 templates

Joins / Keys

- templates.id → joins with:
 - step_options.template_id
 - emergency_rules.template_id
- created_by_admin_id, updated_by_admin_id → FK to admin_users.id

Purpose

Reusable messages; emergency contact details can be written inside template body.

Column	Type	Key	Why
id	UUID	PK	Template id

name	VARCHAR	UNIQUE	UI name
type	ENUM		greeting/fallback/emergency/disclaimer
body	TEXT		Message content (include emergency contact numbers here)
status	ENUM		active/disabled
created_by_admin_id	UUID	FK	Owner
updated_by_admin_id	UUID	FK	Last editor
created_at	TIMESTAMP		Tracking
updated_at	TIMESTAMP		Tracking

4) Emergency Rules (No contacts table)

4.1 emergency_rules

Joins / Keys

- emergency_rules.template_id → FK to templates.id (optional)

Purpose

Defines what keywords trigger emergency behavior and what message to send.

Emergency contact details are stored as plain text inside:

- templates.body OR emergency_contact_text

Column	Type	Key	Why
id	UUID	PK	Rule id
keyword	VARCHAR		Trigger phrase
match_type	ENUM		exact/contains
action_type	ENUM		send_template/send_text/both
template_id	UUID NULL	FK	Which emergency template to send
emergency_contact_text	TEXT NULL		If you want rule-specific numbers (no separate table)

is_active	BOOLEAN	Enable/disable
created_at	TIMESTAMP	Tracking
updated_at	TIMESTAMP	Tracking

✅ How to use it

- If template_id exists → send template body (which contains contacts)
- Else → send emergency_contact_text

5) Caregiver + Conversation History Tables

5.1 caregivers

Joins / Keys

- caregivers.id → joins with:
 - conversations.caregiver_id
- Search by:
 - phone_e164 / email

Column	Type	Key	Why
id	UUID	PK	Caregiver id
full_name	VARCHAR NULL		Name
phone_e164	VARCHAR UNIQUE		Primary identifier
email	VARCHAR NULL	INDEX	Search
status	ENUM		unknown/active/blocked
is_registered	BOOLEAN		Unknown vs completed profile
baby_name	VARCHAR NULL		Baby info
baby_dob	DATE NULL		Optional
notes	TEXT NULL		Admin notes
created_by_admin_id	UUID NULL	FK	Who registered

created_at	TIMESTAMP	Tracking
updated_at	TIMESTAMP	Tracking

5.2 conversations

Joins / Keys

- caregiver_id → FK to caregivers.id (nullable)
- current_flow_id → FK to flows.id (nullable)
- current_step_id → FK to flow_steps.id (nullable)
- messages.conversation_id → FK to conversations.id

Column	Type	Key	Why
id	UUID	PK	Conversation id
caregiver_id	UUID NULL	FK	Link when known
phone_e164	VARCHAR	INDEX	Always keep phone
status	ENUM		open/closed/escalated
current_flow_id	UUID NULL	FK	Current flow
current_step_id	UUID NULL	FK	Progress
last_message_at	TIMESTAMP		Sorting
started_at	TIMESTAMP		Start time
closed_at	TIMESTAMP NULL		Close time

5.3 messages

Joins / Keys

- conversation_id → FK to conversations.id
- optional:
 - flow_id → FK to flows.id
 - step_id → FK to flow_steps.id

Column	Type	Key	Why
id	UUID	PK	Message id
conversation_id	UUID	FK	Thread link
direction	ENUM		inbound/outbound
channel	ENUM		sms
from_phone	VARCHAR		Sender
to_phone	VARCHAR		Receiver
body	TEXT		Message text
provider_message_id	VARCHAR NULL		SNS/Twilio id
status	ENUM		sent/delivered/failed/received
error_code	VARCHAR NULL		Debug
matched_keyword	VARCHAR NULL		Which keyword matched
flow_id	UUID NULL	FK	Context
step_id	UUID NULL	FK	Context
is_emergency	BOOLEAN		Flag
created_at	TIMESTAMP		Timestamp

6) Audit Logs

6.1 audit_logs

Joins / Keys

- audit_logs.admin_user_id → FK to admin_users.id

Column	Type	Key	Why
id	UUID	PK	Log id
admin_user_id	UUID	FK	Who changed

action	VARCHAR	FLOW_PUBLISH etc
object_type	VARCHAR	flow/template/user
object_id	UUID NULL	Target record
before_json	JSON NULL	Optional diff
after_json	JSON NULL	Optional diff
created_at	TIMESTAMP	Time
ip_address	VARCHAR NULL	Security
user_agent	TEXT NULL	Security

7) Final Index List (Performance + Search)

- admin_users.username UNIQUE
- flow_keywords.keyword INDEX
- flows.status INDEX
- flow_steps.flow_id INDEX
- step_options.step_id INDEX
- templates.name UNIQUE
- emergency_rules.keyword INDEX
- caregivers.phone_e164 UNIQUE + INDEX
- caregivers.email INDEX
- conversations.phone_e164 INDEX
- messages.conversation_id INDEX
- messages.created_at INDEX

8) Final Note: Emergency Contacts (Your requirement)

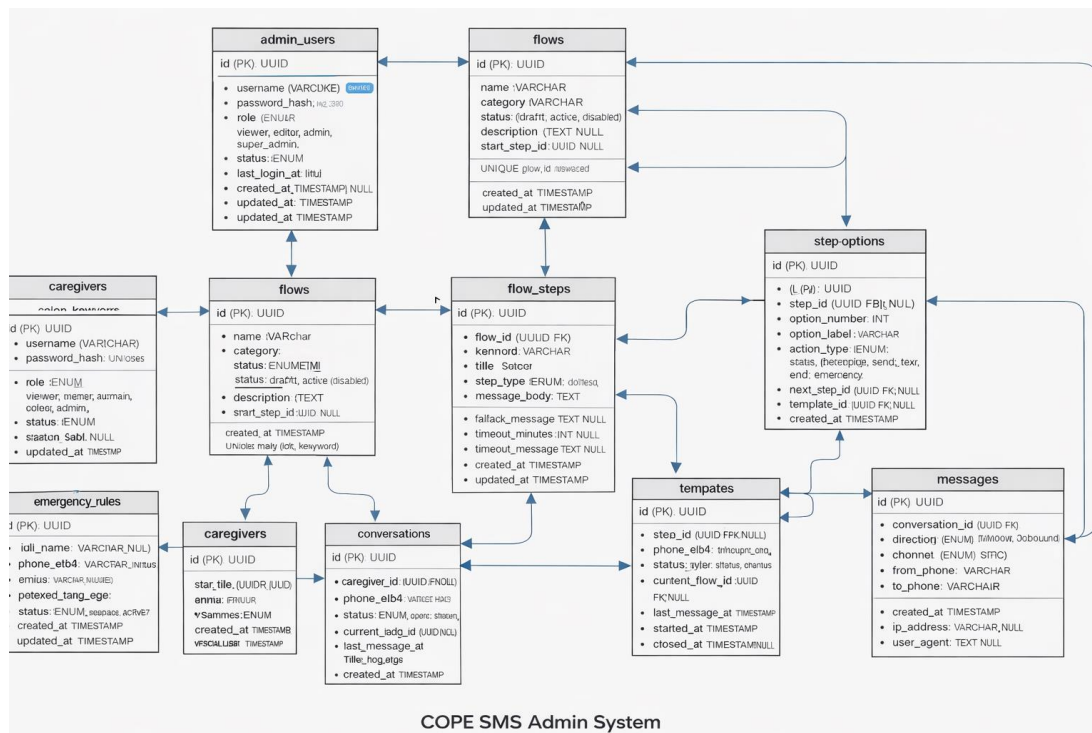
✓ No contacts table needed because:

- Emergency numbers and instructions will be written inside:
 - **Emergency template body** (recommended)

- OR emergency_rules.emergency_contact_text for rule-specific text.

Example template body:

“If baby is not breathing, call 911 immediately. Poison Control: xxx-xxx-xxxx. Local hotline: xxx-xxx-xxxx.”



[

p